Rahul Kumar

AI Software Engineer | Chandigarh, India
+91 94179 20371 | rahulkum94l94@gmail.com | LinkedIn | github.com/rahulCoder9417 | rahulworks.vercel.app

Summary

AI Software Engineer working in **TypeScript, Python, Go, and Rust** on **scalable backends** (job queues, caching, analytics over 300M+ records), **core machine learning** (data preparation, model training and evaluation, embeddings), and **agentic AI** (tool-calling agents, MCP, RAG, LangGraph multi-agent workflows). Owns features end to end, from database schema to UI.

Technical Skills

Languages: TypeScript, Python, Go, Rust, JavaScript, SQL
Backend: Node.js, Hono, Express, Fastify, FastAPI, Fiber, Echo, gRPC, WebSockets, REST, OpenAPI
Agentic AI: Claude API, MCP, LangGraph, LangChain, RAG, Tool Calling, Multi-Agent Workflows, Qdrant, Chroma
Machine Learning: PyTorch, scikit-learn, NumPy, pandas, Hugging Face Transformers, Embeddings, Model Evaluation
Data: PostgreSQL, ClickHouse, Redis, MongoDB, BullMQ, Drizzle, Prisma
Frontend: Next.js, React, Redux Toolkit, Tailwind CSS, shadcn/ui
Infrastructure: Docker, gVisor, GitHub Actions, Caddy, AWS (SES), Prometheus, Grafana, Loki, Tempo
Core Concepts: System Design, Distributed Systems, Data Structures & Algorithms

Experience

InboxKit Jan 2026 – Present
Full-Stack Software Engineer *Remote*

Tech: Go, Fiber, Fastify, Next.js, ClickHouse, Redis, BullMQ, Puppeteer, AWS SES, Docker, MongoDB

- Owned the **Enrich platform** end to end (architecture, frontend, backend, infrastructure, CI/CD), now serving **thousands of daily active users** at **99.9% uptime**, including its REST API and an internal admin dashboard for abuse detection.
- Designed **ClickHouse** analytics over **300M+ records** with multi-tier Redis caching, query rewriting, and partitioning, cutting dashboard query latency by ~**70%** and holding **sub-second p95** under production load.
- Built Go and Node services and automation pipelines (browser automation, transactional email, LinkedIn scraping) on Redis-backed **BullMQ** queues processing **100K+ jobs a day** with retries, backoff, and rate-limited dispatch.
- Built the customer UI in Next.js, Tailwind, and shadcn/ui (dashboards, billing, analytics) and hardened the platform with rate limiting, RBAC, signed sessions, input validation, and audit logging.

Key Projects

Textcrew – Multi-Agent Personal AI Assistant (Live) *TypeScript, Go, gRPC, MCP, BullMQ, gVisor* textcrew.in

- Designed the system and built the backend for a live **AI agent** product people text on Telegram or the web. A lead agent splits each job across **specialist agents** (code, PR review, web research) that run in a per-user **gVisor** sandbox, work in Gmail, GitHub, and Calendar through **MCP** tools, and ask before risky actions.
- Wrote the **Go container manager**, the only service with Docker access, exposing **11 gRPC calls** to create, wake, stop, and evict per-user containers, with idle shutdown, health checks, a drift reconciler, and per-user egress allowlists.
- Built the backend as a TypeScript modular monolith (Hono API + BullMQ worker on Postgres and Redis): webhooks are deduplicated and queued, per-user locks serialize agent turns, and credits run on an append-only ledger with idempotent charges. **188 tests**; all **51 endpoints** are contract-tested against OpenAPI 3.1.

DevSync – Real-Time Collaborative Cloud IDE *Next.js, Yjs, WebSocket, Express, Docker* GitHub

- Built a browser IDE where up to **15 developers edit the same repository at once**, syncing edits, cursors, and selections via **Yjs CRDTs** over WebSockets, with collaboration split from persistence and execution so each scales independently.
- Gave every room its own **Docker** container with isolated disk, a shared terminal, and live preview, running React, Express, and GUI (Tkinter via Xvfb) apps behind a reverse proxy.

GoRedis – Redis-Compatible Database from Scratch *Go, TCP, RESP Protocol* GitHub

- Built a Redis-compatible database in Go with a hand-written **RESP parser** and a goroutine-per-connection server, implementing **40+ commands** (strings with TTL, blocking lists, streams, skip-list sorted sets, geospatial indexes) so the official `redis-cli` and client SDKs connect natively at sub-millisecond latency.
- Implemented **MULTI/EXEC transactions** with `WATCH`-based optimistic concurrency, **master-replica replication** (`PSYNC`/`WAIT`), **RDB snapshots**, an **append-only log**, Pub/Sub, and authentication.

Rust Systems Projects *Rust, TCP, Threads, SDL3* Web Server | 3D Renderer

- A **multi-threaded HTTP server** on raw TCP with a hand-written thread pool that shuts down cleanly when dropped, and a **3D wireframe renderer** with hand-written projection, rotation matrices, and a free-look camera.